memory.md

Skill-Centric Unified Job Search Aggregator System — Project Memory

This is the persistent context file for the project. It exists so that anyone (or any AI agent) picking up work later doesn't have to re-derive decisions already made. Update it whenever a real decision is taken — not for routine code changes, only for things a future session would otherwise have to guess at or re-litigate.

---

1. One-Paragraph Project Identity

A skill-centric job aggregator that ingests postings from ≥3 platforms (API + compliant scraping), deduplicates them across sources, extracts structured skills from job descriptions via NLP, and ranks a personalized feed by semantic similarity between a user's declared skill profile and each job's extracted requirements — replacing keyword search and manual cross-platform comparison. Stack: React/Next.js, FastAPI, PostgreSQL, spaCy + scikit-learn, Celery/Redis, Docker.

---

2. Non-Negotiable Constraints

- Matching is based on the user's **structured skill profile**, not free-text query input. This is the product's core differentiator — do not let it drift back toward plain keyword search.
- Minimum 3 distinct source platforms for the PoC.
- Scraping must always respect `robots.txt` and stay within conservative rate limits — never framed or engineered as a bypass mechanism.
- Out of scope for this phase: multi-lingual parsing, one-click third-party application submission, native mobile apps, employer-facing posting/ATS features.

---

3. Key Architectural Decisions Log

Decision	Rationale	Status
Staged pipeline (Ingestion → Standardization → NLP → Feed) over monolithic design	Workloads are heterogeneous (I/O-bound scraping vs. CPU-bound NLP vs. latency-sensitive reads); isolating stages allows independent scaling/failure handling	Adopted
Staging table before canonical `jobs` write	Prevents upstream format changes from corrupting canonical data; allows quarantine/reprocessing	Adopted
Composite 3-field fuzzy dedup heuristic (company/title/location) + `dedup_hash` fast path	Explainable, tunable, cheap exact-match fast path before expensive fuzzy comparison	Adopted
TF-IDF + cosine similarity for v1 matching (not transformer embeddings)	Lower infra cost for PoC; explicit upgrade path preserved	Adopted; revisit post-PoC
Junction tables (`user_skills` , `job_skills`) enriched with relationship attributes	Only way to represent proficiency/relevance as facts about the *relationship*, not the entities alone	Adopted
Async Phase A (extraction), sync/cacheable Phase B (matching)	Keeps expensive NLP off the request path; only lightweight scoring is synchronous	Adopted
Celery worker pools split: ingestion vs. NLP	Prevents a scraping burst from starving NLP processing, or vice versa	Adopted

---

4. Glossary

Term	Meaning in this project
Canonical record	The single retained `jobs` row after deduplication across sources
Staging table	`staging_jobs` — raw, unvalidated payloads landing zone before standardization
dedup_hash	SHA-256 of normalized (company + title + location); DB-indexed exact-duplicate fast path
Gazetteer	Curated reference list of known skills used by the NER pipeline for exact/fuzzy matching
Relevance score	(`job_skills.relevance_score`) how central a skill is to a *specific job* — NLP-derived
Match score	(`applications.match_score` / feed-time computation) cosine similarity between a *user's* skill vector and a *job's* skill vector
Alternate listing	A duplicate posting's source/URL, preserved after dedup so the user can still apply via their preferred platform

---

5. Open Questions / Risks (update as resolved)

Numeric targets for dedup and NLP precision/recall are referenced in `phase.md` Sprints 2–3 as "agreed targets" but no specific number is fixed yet in the source synopsis — this needs a decision before those sprints close, and the answer should be logged here once set.

Which specific three platforms are the initial ingestion targets (API availability, ToS review) — not fixed in the synopsis; decide and log here before Phase 1 begins.

High-confidence threshold for the future daily match-alert feature is proposed at ~90% in the synopsis but not committed — revisit when Phase 8+ work is scheduled.

---

6. Roadmap Items Not Yet Scheduled

(See `phase.md §8+` for full descriptions — this is just the tracking list, update status as items move.)

- [ ] AI-driven resume parsing (PDF → skill profile)
- [ ] Automated daily match alerts (Celery-scheduled)
- [ ] Source-platform expansion beyond initial 3
- [ ] TF-IDF → transformer embedding migration
- [ ] "Not relevant" feedback signal for supervised tuning
- [ ] Employer-facing analytics dashboard

---

7. Document Map

Doc	Contains
`Architecture.md`	Component breakdown, tech stack, deployment topology, data flow
`design.md`	Full DB schema/DDL, API endpoint catalog, dedup/NLP/matching algorithm design
`phase.md`	Sprint-by-sprint delivery plan with exit criteria
`rules.md`	Hard engineering rules and guardrails (compliance, schema, security, testing)
`memory.md`	This file — decisions, glossary, open questions, roadmap tracking

---

Update this file whenever a real decision is made. It's the project's memory, not its changelog — keep entries decision-shaped, not commit-shaped.